

A COOKBOOK FOR BACKEND DEVELOPERS

Cloud & Kubernetes for ML Cookbook

Run and scale services and models on Kubernetes and the cloud — pods, deployments, autoscaling, GPUs, and managed ML platforms — the infra half of shipping ML.

Kubernetes

Docker

Autoscaling

GPUs

Helm

IaC

SageMaker / Vertex

Cost

Written for a developer who containerizes apps (see the Docker book) and now needs to run and scale them — and models — in production on Kubernetes and the cloud.

Every chapter answers: what it solves, how to do it, the failure modes, and when a simpler platform wins.

Edition 2026 · declare desired state · scale it · don't overpay

Table of Contents

How to Use This Book	4
The one idea: declare desired state, let the system converge	4
What this book covers	5
Part 1 · When to Use Kubernetes (and When Not)	6
1.1 The ladder of deployment options	6
1.2 What Kubernetes genuinely gives you	6
1.3 What it costs	6
Part 2 · The Core Objects: Pods, Deployments, Services	8
2.1 Pod — the unit of running	8
2.2 Deployment — declare N replicas, self-healed	8
2.3 Service — a stable address for shifting pods	9
Part 3 · Configuration: Config, Secrets, Probes	10
3.1 ConfigMaps and Secrets	10
3.2 Health probes — how Kubernetes knows a container is OK	10
3.3 Model-serving needs a startup probe	11
Part 4 · Networking: Services & Ingress	12
4.1 Service types	12
4.2 Ingress — one entry point, HTTP routing	12
4.3 TLS and DNS	13
Part 5 · Scaling	14
5.1 Horizontal Pod Autoscaler (HPA)	14
5.2 Cluster Autoscaler	14
5.3 Horizontal vs vertical	14
Part 6 · GPUs on Kubernetes	16
6.1 Requesting a GPU	16
6.2 The cost problem is the whole game	16
6.3 Training jobs vs serving	17
Part 7 · Serving Models on Kubernetes	18
7.1 The plain-Kubernetes way (start here)	18
7.2 ML-specific serving frameworks	18
7.3 Rollouts and scale-to-zero	19
Part 8 · Infrastructure as Code (Helm & Terraform)	20
8.1 Two layers, two tools	20
8.2 Terraform — declare the cloud	20
8.3 Helm — package your workloads	20
8.4 GitOps (the next step)	21
Part 9 · Cost & Operations	22
9.1 Where the money goes	22
9.2 The cost levers	22
9.3 Observability	22

9.4 Security & operations basics	23
Part 10 · Capstone & Cheat Sheet	24
10.1 Capstone — deploy a scalable model service	24
10.2 kubectl cheat sheet	24
10.3 Object cheat sheet	24
10.4 The five rules.	25

How to Use This Book

You can containerize an app (the Docker book) and serve a model (the MLOps book). This book is the next layer: **running those containers reliably and at scale** on **Kubernetes** and the **cloud** — the infrastructure half of shipping software and ML. It's where "it works on my machine / one server" becomes "it stays up, scales with load, and uses a GPU without falling over."

Kubernetes has a reputation for complexity, much of it earned. So this book teaches the **core you actually use daily** (a small subset), is honest that most projects don't need Kubernetes at all, and always points at the simpler managed option when one fits. The goal is competence with the real tool and the judgment to avoid it when it's overkill.

Each chapter answers four questions:

1. **What problem does this solve?**
2. **How do you do it (YAML / kubectl / cloud)?**
3. **What are the failure modes?**
4. **When is a simpler platform the right call?** (Often.)

The one idea: declare desired state, let the system converge

You don't tell Kubernetes *how* to run things step by step. You **declare the desired state** — "3 replicas of this container, this much CPU/memory, reachable here" — and Kubernetes continuously works to make reality match, restarting crashed containers, rescheduling on dead nodes, and scaling to meet the spec.

DIAGRAM

```

you write:  desired state (YAML)  →  Kubernetes control loop  →  actual state
            "3 replicas, 2 CPU each"    constantly reconciles    (keeps it true)
                                      crash? reschedule. node dies? move it.
  
```

That reconciliation loop is the whole philosophy: **declarative, self-healing infrastructure**. Grasp it and the rest is vocabulary.

ENGINEER'S MENTAL MODEL

Kubernetes is a control system: you give it a target (desired state) and it runs a feedback loop to hold reality there — like a thermostat, not a script. This is why you rarely run imperative commands in production; you edit the declared state and let the cluster converge. It's `docker run` replaced by "here's what I want to always be true," and the cluster makes it so.

What this book covers

When to use Kubernetes (and when not); the **core objects** (pods, deployments, services); **configuration** (config maps, secrets, resources, health probes); **networking** (services, ingress); **scaling** (horizontal/cluster autoscaling); **GPUs** for ML workloads; **serving models** on Kubernetes; **infrastructure as code** (Helm, Terraform); and **cost & operations**. Cloud-agnostic, with managed services noted throughout.

HOW TO READ IT

Read Part 1 first and take it seriously — the honest "do you even need this?" Then Parts 2–5 are the core you'll use constantly (objects → config → networking → scaling). Parts 6–7 are the ML-specific payoff (GPUs, model serving). Parts 8–9 make it maintainable and affordable. Practice on a local cluster (kind, minikube, or k3s) — you don't need a cloud bill to learn.

READ THIS FIRST: YOU PROBABLY DON'T NEED KUBERNETES

Kubernetes is powerful and operationally heavy. If you're running a handful of services or one model endpoint, a **managed platform** — a cloud run/container service, a PaaS, or a managed ML endpoint (SageMaker/Vertex) — will serve you faster, cheaper, and with far less to operate. Kubernetes earns its complexity at **real scale, many services, multi-team, or specific control/portability needs**. Learn it (it's everywhere), but reach for the simplest thing that runs your workload.

Part 1 · When to Use Kubernetes (and When Not)

Kubernetes is the industry standard for orchestrating containers, and it's also routinely used where it isn't needed, costing teams months of complexity. Because this decision matters more than any YAML, it comes first.

1.1 The ladder of deployment options

Option	You manage	Good for
PaaS / serverless containers (Cloud Run, App Runner, Fly, Railway)	almost nothing	most apps & simple model endpoints
Managed ML endpoints (SageMaker, Vertex)	the model only	serving models without infra
Managed Kubernetes (EKS, GKE, AKS)	the workloads (not the control plane)	scale, many services, control
Self-managed Kubernetes	everything	special requirements, on-prem

Climb only as far as you must. Most teams reaching for self-managed Kubernetes wanted managed Kubernetes; most reaching for Kubernetes at all wanted a PaaS.

1.2 What Kubernetes genuinely gives you

- **Self-healing** — crashed containers restart; dead nodes' work reschedules.
- **Scaling** — automatic horizontal scaling on load; cluster grows/shrinks nodes.
- **Declarative, portable infra** — the same manifests run on any cluster, any cloud (no lock-in).
- **A rich ecosystem** — one platform for services, jobs, GPUs, networking, secrets, and the tooling around them.
- **Efficient bin-packing** — many workloads share nodes (incl. expensive GPUs).

1.3 What it costs

- **Operational complexity** — a real distributed system to learn and run; upgrades, networking, RBAC, YAML sprawl.
- **A learning curve** measured in months to real proficiency.
- **Overhead** — worth it at scale, pure cost for three containers.

ENGINEER'S MENTAL MODEL

Kubernetes is an operating system for a cluster — brilliant when you have a fleet of workloads to schedule across many machines, overkill when you have one process to run. Ask "do I have a *fleet* problem?" If you're orchestrating many services and jobs across nodes with real scale, yes. If you're running one API, a PaaS is the right OS and Kubernetes is a datacenter you're renting to run Notepad.

WHEN NOT TO USE KUBERNETES

Skip it when: you have a **few services** (a PaaS/serverless-containers platform runs them with near-zero ops); you just need to **serve one model** (a managed ML endpoint or a container service); you're a **small team** without platform engineers to operate a cluster; or a managed option meets your scale. Choosing Kubernetes for these trades weeks of your time for capabilities you won't use. The best Kubernetes decision is often not to.

COMMON MISTAKE

Adopting Kubernetes for a small app because it's the "professional" choice, then spending more effort operating the cluster than building the product — YAML, upgrades, networking debugging, and a bill for idle nodes. Start on a managed platform; migrate to Kubernetes when you can name the specific scaling or control need it solves that your platform can't. Resume-driven infrastructure is a real and expensive mistake.

EXERCISE 1.1

Take three workloads (an API, a batch job, a model endpoint). For each, pick the *lowest* rung that runs it: PaaS/serverless, managed ML endpoint, managed Kubernetes, or self-managed. Justify each in a sentence. If none clearly needs Kubernetes, that's the correct and common answer — and you'll still learn it in the rest of this book for when one does.

Part 2 · The Core Objects: Pods, Deployments, Services

Kubernetes has dozens of object types; you use a handful daily. This chapter is that handful — **pods**, **deployments**, and **services** — which together run and expose a scalable app. Learn these three and you can deploy real workloads.

2.1 Pod — the unit of running

A **pod** is the smallest deployable unit: one (or a few tightly-coupled) containers sharing a network and storage. You rarely create pods directly — they're ephemeral and disposable, created *for* you by a deployment.

DIAGRAM

Pod = one running instance of your container (+ its sidecars, if any)
ephemeral: it can die/move at any time – never treat a pod as permanent

2.2 Deployment — declare N replicas, self-healed

A **deployment** declares "run N identical replicas of this container, and keep them running." It creates/replaces pods to match, handles rolling updates, and self-heals. This is what you actually write for a service:

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: model-api }
spec:
  replicas: 3                                # desired state: always 3
  selector: { matchLabels: { app: model-api } }
  template:
    metadata: { labels: { app: model-api } }
    spec:
      containers:
        - name: api
          image: registry/model-api:1.4.0    # pin a version, never :latest
          ports: [{ containerPort: 8000 }]
          resources:
            requests: { cpu: "500m", memory: "512Mi" } # scheduler guarantees these
            limits:   { cpu: "1",    memory: "1Gi" }   # capped here
```

`kubectl apply -f deploy.yaml` and Kubernetes makes it so; `kubectl scale deploy/model-api --replicas=5` or edit the YAML to change the desired count.

2.3 Service — a stable address for shifting pods

Pods come and go with changing IPs, so you never talk to a pod directly. A **service** gives a stable name and IP that load-balances across the current pods:

YAML

```
apiVersion: v1
kind: Service
metadata: { name: model-api }
spec:
  selector: { app: model-api }      # targets pods with this label
  ports: [{ port: 80, targetPort: 8000 }]
```

Now anything in the cluster reaches the app at `model-api:80`, regardless of which pods exist.

ENGINEER'S MENTAL MODEL

Map it to what you know: a **pod** is a running container instance, a **deployment** is a replica set + supervisor that keeps N of them alive (like a process manager that respawns crashes and does rolling deploys), and a **service** is a stable load-balanced DNS name in front of them (the internal load balancer). Deployment = "keep it running and updated"; Service = "give it a reliable address."

COMMON MISTAKE

Treating pods as pets — creating them directly, or storing important state on a pod's local disk. Pods are cattle: ephemeral, replaceable, killed and moved whenever the scheduler needs to. Run apps via **deployments** (so replacements are automatic) and keep state in external stores (databases, object storage) or dedicated stateful primitives — never on a pod that can vanish mid-request.

PRODUCTION TIP

Always set resource **requests and limits** (above). Requests tell the scheduler how much to reserve (so pods land on nodes with room); limits cap usage (so one pod can't starve neighbors). Omit them and you get mysterious OOM kills, noisy-neighbor slowdowns, and pods that won't schedule — the most common source of "why is my cluster misbehaving?"

EXERCISE 2.1

On a local cluster (kind/minikube/k3s), deploy a containerized app with a Deployment (3 replicas, resource requests/limits) and expose it with a Service. `kubectl delete` one of its pods and watch the deployment recreate it (self-healing). Scale to 5 replicas by editing the YAML and re-applying. Confirm the Service load-balances across all current pods.

Part 3 · Configuration: Config, Secrets, Probes

A container image should be built once and run in any environment; what changes between dev and prod is **configuration**. Kubernetes separates config and secrets from the image, and uses **health probes** to know when a container is actually ready. Getting these right is most of the difference between flaky and solid deployments.

3.1 ConfigMaps and Secrets

- **ConfigMap** — non-sensitive config (feature flags, URLs, tuning) injected as env vars or files.
- **Secret** — sensitive values (API keys, DB passwords), same idea but handled as secret data (base64-encoded, access-controlled; encrypt at rest — see the tip).

YAML

```
envFrom:
  - configMapRef: { name: model-api-config }
  - secretRef:    { name: model-api-secrets }
```

This is the 12-factor rule (your Python config chapter): **config comes from the environment, not the image**, so the same image promotes cleanly dev → staging → prod.

3.2 Health probes — how Kubernetes knows a container is OK

Kubernetes can only self-heal if it knows what "healthy" means. You tell it with probes:

- **Liveness** — "is it alive?" Fails → Kubernetes restarts the container (recovers a hung process).
- **Readiness** — "can it serve traffic *now*?" Fails → pulled from the Service's load balancer (but not restarted). Vital during startup and warm-up.
- **Startup** — grace period for slow-booting apps before liveness kicks in (model loading can be slow!).

YAML

```
readinessProbe:
  httpGet: { path: /health, port: 8000 }
  initialDelaySeconds: 5
  periodSeconds: 10
livenessProbe:
```

```
httpGet: { path: /health, port: 8000 }
periodSeconds: 15
```

ENGINEER'S MENTAL MODEL

Probes are the health checks you already wrote (your Docker healthcheck, your load balancer's checks) made first-class: **readiness** gates traffic (like a load balancer only routing to healthy backends), **liveness** restarts the hung ones (like a process supervisor). ConfigMaps/Secrets are your `.env` and secret manager, decoupled from the image. Nothing new conceptually — just the Kubernetes-native way to express it.

3.3 Model-serving needs a startup probe

ML containers often take tens of seconds to load a model into memory before they can serve. Without a **startup probe** (or a generous `initialDelaySeconds`), liveness checks fail during that load and Kubernetes kills the container mid-boot — a restart loop that never lets the model finish loading.

COMMON MISTAKE

A liveness probe with too short a delay on a slow-starting (model-loading) container: it's killed and restarted before it ever finishes booting, looping forever and never serving. Use a **startup probe** (or ample `initialDelay`) for slow boots, and keep **readiness** and **liveness** distinct — conflating them (restarting a pod that's merely still warming up) causes cascading restarts under load exactly when you need stability.

PRODUCTION TIP

Kubernetes Secrets are only base64-encoded by default, **not encrypted** — treat them as "kept out of the image," not "secure." Enable **encryption at rest** for secrets, restrict access with RBAC, and for anything serious use an external secret manager (Vault, cloud secret managers) via a secrets operator. Assuming `kind: Secret` means "encrypted" is a common and dangerous misconception.

EXERCISE 3.1

Externalize a deployment's config into a ConfigMap and its keys into a Secret, injected as env vars — rebuild nothing, just re-apply. Add readiness and liveness probes hitting `/health`. Then simulate a slow model load (sleep on startup) and watch a too-tight liveness probe cause a restart loop; fix it with a startup probe.

Part 4 · Networking: Services & Ingress

Getting traffic to your pods — from inside the cluster and from the outside world — is Kubernetes networking. You need just a few concepts: the service types, and **ingress** for HTTP routing. This is enough to expose an API or a model endpoint properly.

4.1 Service types

A Service (Part 2) has a type that decides who can reach it:

Type	Reachable from	Use for
ClusterIP (default)	inside the cluster only	internal services (most things)
NodePort	a port on every node	dev/testing, rarely prod
LoadBalancer	the internet (cloud LB)	exposing one service externally

Internal-to-internal calls use ClusterIP by DNS name (`model-api.default.svc`). For external access you *could* give every service a LoadBalancer — but that's one cloud load balancer (and bill) each, which is where ingress comes in.

4.2 Ingress — one entry point, HTTP routing

An **Ingress** (with an ingress controller like nginx or Traefik) is a single external entry point that routes HTTP by host/path to many internal services — one load balancer, TLS termination, and routing rules in one place:

YAML

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: api-ingress }
spec:
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /models      # api.example.com/models → model-api service
            pathType: Prefix
            backend: { service: { name: model-api, port: { number: 80 } } }
          - path: /app         # api.example.com/app → web-app service
            pathType: Prefix
            backend: { service: { name: web-app, port: { number: 80 } } }
```

ENGINEER'S MENTAL MODEL

Ingress is nginx/Traefik as a reverse proxy — the same host/path routing and TLS termination you'd configure by hand (your n8n/self-hosting reverse-proxy notes), expressed declaratively and wired to services by label. ClusterIP is a private internal DNS name; LoadBalancer is a public front door per service; Ingress is *one* front door that fans out to many — almost always what you want for HTTP.

4.3 TLS and DNS

Ingress terminates TLS (certs via cert-manager + Let's Encrypt, commonly), so your services speak plain HTTP internally while the outside gets HTTPS. Point your domain's DNS at the ingress's load balancer and you have `https://api.example.com` routing to the right pods.

COMMON MISTAKE

Giving every service its own `LoadBalancer` — each spins up a separate cloud load balancer with its own IP and monthly cost, and you manage TLS/DNS N times. Use **one Ingress** in front of many ClusterIP services: one load balancer, one place for TLS and routing. Reserve dedicated `LoadBalancer` services for the few non-HTTP cases that need them.

WHEN NOT TO HAND-ROLL INGRESS

On a managed platform or PaaS, external routing and TLS are often provided — you don't need to run an ingress controller and cert-manager yourself. And for a single public service, a cloud LB or the platform's built-in routing may be simpler than standing up ingress. Use ingress when you're routing **multiple HTTP services** through one entry point on your own cluster.

EXERCISE 4.1

On a local cluster, install an ingress controller (nginx). Deploy two services (e.g. a model API and a web app) as ClusterIP, then an Ingress routing `/models` and `/app` to them by path. Hit both through the single ingress entry point. Bonus: add local TLS and confirm HTTPS terminates at the ingress while services stay HTTP.

Part 5 · Scaling

One of Kubernetes' biggest payoffs: it scales your workloads with demand, automatically. Two layers work together — scaling **pods** (more replicas) and scaling **nodes** (more machines). Understanding both, and their limits, is what lets you handle load without over-paying for idle capacity.

5.1 Horizontal Pod Autoscaler (HPA)

The HPA adds/removes pod replicas based on a metric (CPU, memory, or custom like requests/sec or queue depth):

YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: model-api }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: model-api }
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } }
```

Load rises → CPU passes 70% → more replicas; load falls → scale back to `min`. Horizontal (more replicas) is the default and best-behaved scaling for stateless services.

5.2 Cluster Autoscaler

Pods only run if there's node capacity. The **cluster autoscaler** adds nodes when pods can't be scheduled (pending for lack of resources) and removes underused nodes — so the *cluster itself* grows and shrinks with demand. HPA scales pods; the cluster autoscaler scales the machines under them. You need both for true elasticity.

DIAGRAM

```
load ↑ → HPA adds pods → no room? → cluster autoscaler adds nodes → pods schedule
load ↓ → HPA removes pods → nodes idle → cluster autoscaler removes nodes → bill ↓
```

5.3 Horizontal vs vertical

- **Horizontal** (more replicas) — the default; great for stateless services, and how you handle traffic.

- **Vertical** (bigger pods) — for workloads that can't parallelize; a single large model that needs more memory scales up, not out.

ENGINEER'S MENTAL MODEL

This is autoscaling groups + a load balancer, made native and two-layered: HPA is "add app instances behind the LB when busy," cluster autoscaler is "add VMs when the instances need somewhere to run." The mental shift from a fixed fleet is designing **stateless** workloads so *more replicas* actually helps — state on a pod (Part 2) breaks horizontal scaling.

COMMON MISTAKE

Configuring an HPA but forgetting the cluster autoscaler (or fixed node capacity), so under load the HPA wants 15 pods, there's only room for 6, and the rest sit **Pending** forever — you "have autoscaling" but it silently can't scale. Enable both layers, and set resource requests accurately (Part 2) so the schedulers can reason about capacity. Autoscaling is only as good as the layer beneath it.

WHEN NOT TO AUTOSCALE (OR SCALE OUT)

Don't add an HPA to a workload that can't scale horizontally — a stateful singleton, or a job needing exclusive access. Scale those vertically or rethink the design. And for steady, predictable load, a fixed replica count is simpler and cheaper than autoscaling machinery. Autoscale variable, stateless workloads; don't bolt it onto everything.

EXERCISE 5.1

Add an HPA to your model-api deployment (CPU target 70%, min 2, max 10). Load- test it (hey/k6) and watch replicas scale up, then down as load drops. On a cloud cluster, enable the cluster autoscaler and drive load high enough that the HPA needs more nodes — watch nodes get added, then removed when idle. Note what happens if you *disable* the cluster autoscaler and pods go Pending.

Part 6 · GPUs on Kubernetes

Training and serving deep-learning models often need GPUs, and GPUs are expensive and scarce — so running them well on Kubernetes is a distinct skill. This chapter covers requesting GPUs, sharing them, and the cost discipline that keeps a GPU cluster from becoming a runaway bill.

6.1 Requesting a GPU

With the NVIDIA device plugin installed on the cluster, a pod requests GPUs as a resource, like CPU or memory:

YAML

```
spec:
  containers:
    - name: trainer
      image: registry/trainer:1.2.0
      resources:
        limits:
          nvidia.com/gpu: 1          # this pod gets 1 GPU
      nodeSelector:
        cloud.google.com/gke-accelerator: nvidia-tesla-t4  # (cloud-specific) pick GPU
type
```

The scheduler places the pod on a node with an available GPU. GPU nodes are a separate, pricier node pool you add to the cluster.

6.2 The cost problem is the whole game

GPUs cost many times what CPU nodes do and are often **idle** between jobs. The levers that matter:

- **Separate GPU node pool with autoscaling to zero** — scale GPU nodes down to *none* when no GPU work is queued, so you pay only while training/serving.
- **Bin-pack** — let multiple smaller workloads share GPU nodes rather than one job hogging a node at 10% utilization.
- **Spot/preemptible GPUs** for interruptible training — a large discount if your job can checkpoint and resume.
- **Batch/queue GPU jobs** so expensive hardware stays busy, not idle-but-on.

ENGINEER'S MENTAL MODEL

A GPU is a metered, scarce, expensive resource — treat it like one. The instinct is the opposite of "always-on web server": you want GPU capacity to exist *only* while it's doing paid work, then vanish. Scale-to-zero GPU pools, spot instances for training, and packing jobs onto busy GPUs are the difference between a reasonable bill and a shocking one. Idle GPUs are money on fire.

6.3 Training jobs vs serving

- **Training** — a **Job** (run-to-completion), ideally on spot GPUs with checkpointing so a preemption resumes rather than restarts. Scale the pool to zero when the queue's empty.
- **Serving** — a **Deployment** on GPU nodes; here you *do* need it always-on for latency, so right-size the GPU, consider quantization (your DL/FT books) to fit a cheaper GPU, and scale replicas to real traffic.

COMMON MISTAKE

Leaving GPU nodes running 24/7 for occasional training, or provisioning a big always-on GPU deployment sized for peak that sits near-idle most of the day. GPU idle time is the dominant cost in ML infra. Use **scale-to-zero** GPU pools for training and **autoscaled, right-sized** GPU serving — and quantize models to fit smaller/cheaper GPUs where quality allows. Measure GPU *utilization*, not just that jobs finished.

WHEN NOT TO RUN GPUS ON YOUR OWN CLUSTER

For occasional training or a single model endpoint, a **managed** option — a cloud training job, a serverless GPU service, or a managed inference endpoint (SageMaker/Vertex) — is often cheaper and far less work than operating a GPU node pool, drivers, and the device plugin yourself. Run GPUs on your cluster when you have **sustained, high GPU utilization** and need the control; otherwise rent them per-use.

EXERCISE 6.1

(Cloud cluster.) Add a GPU node pool with autoscaling to zero. Run a training **Job** that requests 1 GPU, and confirm a node spins up for it and is removed after. Deploy a GPU-backed model-serving Deployment and load-test it. Then compare the estimated monthly cost of your GPU setup against a managed inference endpoint for the same workload — and decide which you'd actually use.

Part 7 · Serving Models on Kubernetes

This is where the MLOps and Kubernetes books meet: taking a model service and running it on a cluster with autoscaling, health, and rollouts. You can do it with the plain objects you already know, or with ML-specific serving frameworks that add model-aware features. This chapter is the practical bridge.

7.1 The plain-Kubernetes way (start here)

A model API is just a container, so the objects from Parts 2-5 fully apply:

DIAGRAM

```
Deployment (your FastAPI model server, N replicas, GPU if needed)
+ resource requests/limits (Part 2)
+ readiness/liveness/startup probes – startup probe for model load! (Part 3)
+ Service + Ingress for access (Part 4)
+ HPA for autoscaling on load (Part 5)
= a scalable, self-healing model endpoint, no special tooling
```

For most single models this is all you need. Load the model from your registry (MLOps book) at startup, serve with FastAPI, and let Kubernetes handle scaling and healing.

7.2 ML-specific serving frameworks

At scale or with many models, model-aware servers add real value over a bare Deployment:

- **KServe / Seldon** — Kubernetes-native model serving: autoscaling (incl. scale-to-zero), canary rollouts, and multi-framework support as first-class.
- **Triton / TorchServe** — high-performance inference servers with **dynamic batching** (combine incoming requests to use the GPU efficiently) and multi-model hosting.
- **vLLM** — high-throughput LLM serving (your Fine-Tuning book), deployable on Kubernetes.

These solve model-serving problems (batching, GPU efficiency, many-model hosting, canaries) that you'd otherwise hand-build.

ENGINEER'S MENTAL MODEL

Serving a model on Kubernetes is serving any stateless service you've deployed — Deployment + Service + Ingress + HPA — with two ML twists: a **startup probe** for slow model loading, and **request batching** to use a GPU well. Reach for KServe/Triton when those twists (plus scale-to-zero and canaries) become enough work that a purpose-built server beats maintaining your own. Start plain; adopt frameworks when the plain version strains.

7.3 Rollouts and scale-to-zero

- **Canary/rolling updates** — deploy a new model version to a fraction of traffic, watch metrics (MLOps monitoring), then ramp — Kubernetes rolling updates or KServe canaries.
- **Scale-to-zero** — for spiky or low-traffic model endpoints (especially GPU), KServe/Knative can scale replicas to zero and cold-start on request, so idle models cost nothing.

COMMON MISTAKE

Reaching for a heavyweight serving framework (KServe/Seldon/Triton) for a single low-traffic model — now you operate the framework *and* the cluster for one endpoint a plain Deployment would have served. Start with Deployment + Service + HPA + a startup probe; graduate to a serving framework when you specifically need dynamic batching, scale-to-zero, many-model hosting, or built-in canaries.

WHEN NOT TO SERVE ON KUBERNETES AT ALL

If you're not already running Kubernetes for other reasons, serving one model on it is a lot of setup — a **managed inference endpoint** (SageMaker/Vertex) or a serverless container/GPU service will get you a scalable endpoint far faster. Serve on Kubernetes when it's already your platform, or when you need the control and portability at real scale.

EXERCISE 7.1

Deploy a registered model (MLOps book) to your cluster the plain way: Deployment (with a startup probe for model load) + Service + Ingress + HPA; load-test and watch it scale. Then deploy the same model with KServe and compare — what did the framework give you (scale-to-zero? canary? batching?) and was it worth the extra moving parts for your traffic level?

Part 8 · Infrastructure as Code (Helm & Terraform)

Clicking in a cloud console and running `kubectl apply` by hand doesn't scale and isn't reproducible. **Infrastructure as code** — declaring your cluster and its workloads in version-controlled files — brings the discipline you already apply to application code to your infrastructure. Two tools dominate, at two layers.

8.1 Two layers, two tools

Tool	Manages	Analogy
Terraform (or OpenTofu / Pulumi)	the cloud infra — the cluster, node pools, networks, databases	provisioning the machines
Helm	the workloads on the cluster — your app's bundle of manifests	packaging the app

Terraform builds the cluster; Helm deploys apps onto it. They're complementary.

8.2 Terraform — declare the cloud

HCL

```
resource "google_container_cluster" "primary" {
  name     = "ml-cluster"
  location = "us-central1"
  # ... node pools (incl. a GPU pool), networking, etc.
}
```

`terraform apply` creates or updates the real infrastructure to match the files. The cluster, its node pools, networking, and cloud databases become reviewable, versioned code — and reproducible in another region or account.

8.3 Helm — package your workloads

Raw manifests duplicate a lot across environments (dev/staging/prod differ only in a few values). A **Helm chart** is a templated, parameterized bundle of your Kubernetes objects with per-environment values:

SHELL

```
helm install model-api ./chart -f values.prod.yaml    # same chart, prod values
helm upgrade model-api ./chart -f values.prod.yaml    # roll out a change
```

```
helm rollback model-api
release
```

```
# revert to the previous
```

One chart, many environments via values files; versioned releases with easy rollback. It's also how you install third-party components (ingress controllers, monitoring stacks) in one command.

ENGINEER'S MENTAL MODEL

Terraform is your dependency-and-environment manifest for the *cloud* (declare what infra exists, apply to converge); Helm is a package manager + templating for your *Kubernetes apps* (like npm + a template engine — one parameterized package, per-env values, versioned installs, rollbacks). Both replace click-ops and hand-run commands with reviewable, repeatable, version-controlled files — the same reason you don't deploy code by FTP.

8.4 GitOps (the next step)

Teams often go further: the Git repo is the **single source of truth**, and a controller (Argo CD, Flux) continuously syncs the cluster to match it. You deploy by merging a PR; the cluster converges. It's the reconciliation philosophy (Part 0) extended to your whole delivery flow.

COMMON MISTAKE

Building the cluster and deploying by hand — console clicks and ad-hoc `kubectl apply` — so no one can reproduce the environment, there's no history of what changed, and staging and prod have silently drifted apart. Put **infra in Terraform and apps in Helm from the start**; a hand-built cluster is an un-versioned production server that only its creator understands, and it will bite you at the worst time.

WHEN NOT TO INVEST HEAVILY IN IAC

For a quick experiment or a throwaway cluster you'll delete tomorrow, full Terraform + Helm + GitOps is overkill — a script or a few commands is fine. But the moment infrastructure is **long-lived, shared, or production**, IaC stops being optional. Match the rigor to the lifespan: throwaway can be imperative; anything that persists should be code.

EXERCISE 8.1

Package your model-api (Deployment + Service + Ingress + HPA) as a Helm chart with a `values.yaml`, and install it with different values for "dev" (1 replica) and "prod" (3 replicas) — same chart. Practice `helm upgrade` to roll out a change and `helm rollback` to revert. Bonus: define a local (kind) cluster or a cloud node pool in Terraform and apply it.

Part 9 · Cost & Operations

A Kubernetes cluster is easy to run *expensively* and easy to run *blind*. This chapter is the operational discipline that keeps it affordable and observable — the part that turns "we have a cluster" into "we run it well." Cost, especially, is where ML infrastructure quietly hemorrhages money.

9.1 Where the money goes

- **Idle nodes** — capacity provisioned for peak, running at 3am with nothing on it.
- **Idle GPUs** — the big one (Part 6); a forgotten GPU node dwarfs everything else.
- **Over-requested resources** — pods reserving far more CPU/memory than they use, so the cluster needs more nodes than the real work requires.
- **Unbounded autoscaling** — no `maxReplicas` /node cap, so a traffic spike (or a bug) scales into a huge bill.
- **Orphaned resources** — old load balancers, volumes, and dev clusters nobody deleted.

9.2 The cost levers

- **Cluster autoscaler + scale-to-zero** (Parts 5-6) — pay for capacity only when used, especially GPUs.
- **Right-size requests** — set requests to real usage (measure it); over-requesting wastes whole nodes.
- **Spot/preemptible nodes** for fault-tolerant work (batch, training) — large discounts.
- **Caps** — `maxReplicas` and node-pool maxima so autoscaling can't run away.
- **Cost visibility** — tools like Kubecost, or cloud cost dashboards, tagged by team/workload, so spend is attributable and watched.

ENGINEER'S MENTAL MODEL

Cost optimization is capacity planning as a feedback loop: measure actual usage, right-size requests to it, scale capacity to zero when idle, buy discounted (spot) capacity for interruptible work, and cap the maximum. It's the same profile-then-optimize discipline you'd apply to performance — except the metric is dollars, and idle GPUs are the hottest line in the profile.

9.3 Observability

You can't operate (or right-size) what you can't see. The standard stack:

- **Metrics** — Prometheus (cluster + app metrics) with Grafana dashboards.
- **Logs** — centralized aggregation (Loki, ELK, or a cloud logging service).

- **Traces** — OpenTelemetry for request flows across services.
- **Alerts** — on failures, resource pressure, cost anomalies, and (for models) the drift/quality signals from the MLOps book.

9.4 Security & operations basics

RBAC (least-privilege access to the cluster), network policies (which pods can talk to which), keeping Kubernetes and images patched, and — the boring one that matters — **backups and a tested restore** for stateful data. A cluster you can't recover isn't production-ready.

COMMON MISTAKE

Running with no cost visibility and no caps, then getting a shock bill from idle GPU nodes, over-provisioned requests, and unbounded autoscaling — often more than the workload could ever justify. Put in **cost monitoring, right-sized requests, scale-to-zero, spot instances, and hard caps** early; on GPU clusters especially, cost discipline isn't optional, it's the main operational skill.

WHEN NOT TO RUN YOUR OWN CLUSTER (THE RECURRING THEME)

If operating all of this — autoscaling, observability, security, cost, backups, upgrades — is more than your team can sustain, that's the signal a **managed platform** was the right call. There's no prize for self-operating infrastructure a managed service would run better and cheaper. Kubernetes competence includes knowing when the answer is "let someone else run it."

EXERCISE 9.1

Audit a cluster (real or local) for cost and observability: check pods' requested vs actual resource usage and right-size an over-requested one; add `maxReplicas` caps to your HPAs; stand up Prometheus + Grafana and build a dashboard of node utilization and app latency; and add an alert on a pod restart loop. List the three biggest cost levers for *your* setup.

Part 10 · Capstone & Cheat Sheet

10.1 Capstone — deploy a scalable model service

Take a containerized model (Docker + MLOps books) to a running, scalable, observable Kubernetes service:

1. **Decide** it honestly — is Kubernetes the right rung, or would a PaaS/managed endpoint do? (Part 1). Proceed to learn, but note the simpler option.
2. **Deploy** — a Deployment (resource requests/limits, a **startup probe** for model load) + Service (Parts 2–3).
3. **Expose** — an Ingress with TLS routing to the service (Part 4).
4. **Scale** — an HPA (plus cluster autoscaler on cloud); load-test it (Part 5).
5. **GPU** (if needed) — a scale-to-zero GPU node pool; right-sized serving (Part 6).
6. **Package** — as a Helm chart with dev/prod values; practice upgrade + rollback (Part 8).
7. **Operate** — Prometheus/Grafana dashboards, alerts, `maxReplicas` caps, and a cost check (Part 9).

Right-size it: plain Deployment+Service+HPA serves most models; add KServe/Triton (Part 7) only when batching/scale-to-zero/canaries justify it.

10.2 kubectl cheat sheet

DIAGRAM

<code>kubectl apply -f x.yaml</code>	create/update from manifest (declarative)
<code>kubectl get pods deploy svc hpa</code>	list resources (add -A for all namespaces)
<code>kubectl describe pod <p></code>	events + status (first stop for "why won't it run?")
<code>kubectl logs <pod> [-f]</code>	container logs (follow with -f)
<code>kubectl exec -it <pod> -- sh</code>	shell into a container
<code>kubectl scale deploy/x --replicas=5</code>	manual scale
<code>kubectl rollout status undo deploy/x</code>	watch / roll back a deployment
<code>kubectl top pods nodes</code>	actual CPU/mem usage (right-sizing)
<code>kubectl get events --sort-by=.lastTimestamp</code>	what just happened

10.3 Object cheat sheet

DIAGRAM

Pod	one running container instance (ephemeral – never a pet)
Deployment	keeps N replicas alive; rolling updates; self-heals
Service	stable address + load balancing across pods (ClusterIP default)
Ingress	one HTTP entry point; host/path routing + TLS (nginx/Traefik)
ConfigMap/Secret	config & secrets injected as env/files (not baked in image)
Probes	readiness (gate traffic) · liveness (restart) · startup (slow boot)
HPA	autoscale pod replicas on a metric (min/max)
Cluster autoscaler	add/remove NODES so pods can schedule

Job	run-to-completion (training, batch)
Helm chart	templated, versioned package of manifests (per-env values)

10.4 The five rules

1. **You probably don't need Kubernetes.** Use the lowest rung — PaaS, managed endpoint, managed k8s — that runs your workload.
2. **Declare desired state; let it converge.** Edit YAML and `apply`; don't hand-run imperative changes in prod.
3. **Set requests/limits and the right probes** — especially a startup probe for model loading. Most cluster pain traces here.
4. **Scale both layers, with caps.** HPA (pods) + cluster autoscaler (nodes); scale GPUs to zero; bound the max.
5. **Watch cost and health.** Idle GPUs and over-requests are the bill; monitor, right-size, and cap — or let a managed platform do it.

YOU'RE READY

You can now run and scale services and models on Kubernetes — core objects, config and probes, networking, autoscaling, GPUs, model serving, Helm/Terraform, and cost/ops — the infrastructure half of shipping ML at scale. Paired with the MLOps book, you can take a model from training to a self-healing, autoscaling, monitored production endpoint. And you've learned the most valuable judgment of all here: reach for this power only when the workload truly needs it, and let a managed platform carry the rest.